

# Relatório Técnico — Tech Challenge Fase 2

Otimização de Modelos de Diagnóstico de AVC  
via Algoritmos Genéticos e Integração com LLM

FIAP Pós-Tech — IA para Devs

Data: 14/07/2026

---

Projeto 1 — Otimização de Modelos de Diagnóstico

# 1. Introdução e Contexto

Este relatório documenta o Tech Challenge da Fase 2 do curso de pós-graduação Pós-Tech IA para Devs da FIAP. O objetivo é aprimorar os modelos de previsão de risco de AVC desenvolvidos na Fase 1, aplicando Algoritmos Genéticos para otimização de hiperparâmetros e integrando um LLM para geração de explicações em linguagem natural.

## 1.1 O Problema

O AVC é uma das principais causas de morte e incapacidade no mundo. Ferramentas de triagem baseadas em machine learning podem auxiliar profissionais de saúde na identificação precoce de pacientes em risco. Utilizamos o Stroke Prediction Dataset do Kaggle (5.110 registros, 11 features, 4,9% de casos positivos).

## 1.2 Resultados da Fase 1 — Linha de Base

O **Recall** é a métrica principal: falsos negativos (AVC não detectado) têm maior impacto clínico do que falsos positivos.

| Modelo              | Accuracy | Precision | Recall | F1-Score |
|---------------------|----------|-----------|--------|----------|
| Regressão Logística | 83.37%   | 13.41%    | 44.00% | 0.2056   |
| Random Forest       | 92.86%   | 25.53%    | 24.00% | 0.2474   |

## 1.3 Motivação para Otimização

A busca manual de hiperparâmetros é custosa e subótima. Os Algoritmos Genéticos oferecem busca heurística eficiente sem requerer gradiente ou pressupostos sobre a superfície de otimização, sendo adequados para espaços mistos (contínuos + categóricos).

## 2. Algoritmo Genético para Otimização de Hiperparâmetros

O motor de otimização foi implementado com **DEAP** (Distributed Evolutionary Algorithms in Python), utilizando tipos nativos de indivíduo e fitness.

### 2.1 Representação dos Cromossomos

**Logistic Regression — 4 genes:**

| Gene       | Tipo  | Espaço de Busca                           |
|------------|-------|-------------------------------------------|
| C          | float | [0.001, 100.0]                            |
| solver_idx | int   | 0–3 → {lbfgs, liblinear, saga, newton-cg} |
| max_iter   | int   | [100, 1000]                               |
| cw_idx     | int   | 0–1 → {None, 'balanced'}                  |

**Random Forest — 5 genes:**

| Gene              | Tipo | Espaço de Busca                    |
|-------------------|------|------------------------------------|
| n_estimators      | int  | [50, 500]                          |
| max_depth_idx     | int  | 0 → None; 1–28 → profundidade 3–30 |
| min_samples_split | int  | [2, 20]                            |
| min_samples_leaf  | int  | [1, 10]                            |
| cw_idx            | int  | 0–1 → {None, 'balanced'}           |

### 2.2 Função Fitness

Avaliada com CV estratificado (2 folds) exclusivamente sobre os dados de treino, evitando vazamento de dados:

```
fitness = 0.6 × recall_cv + 0.4 × f1_cv - 0.15 (penalidade se recall_cv < 0.30)
```

### 2.3 Operadores Genéticos

| Operador               | Implementação                                                           |
|------------------------|-------------------------------------------------------------------------|
| Seleção                | Torneio (tournsize=3) via <code>deap.tools.selTournament</code>         |
| Crossover              | Single-point ( <code>deap.tools.cxOnePoint</code> ) — taxa padrão: 0.80 |
| Mutação float          | Gaussiana com $\sigma \propto$ intervalo do gene + clamp                |
| Mutação int categórico | Troca aleatória entre categorias válidas                                |
| Mutação int numérico   | Passo gaussiano arredondado + clamp                                     |

### 2.4 Loop Principal e Critério de Parada

O loop segue o ciclo: avaliar fitness → registrar histórico → verificar convergência (patience + min\_delta) → selecionar elites → torneio → crossover → mutação → reavaliar indivíduos inválidos → nova população = elites + offspring. O elitismo garante monotonia do melhor fitness entre gerações.

### 2.5 Experimentos Realizados

| Experimento | Modelo              | Pop. | Ger. | Mut. | Crossover |
|-------------|---------------------|------|------|------|-----------|
| EXP-01      | Logistic Regression | 20   | 30   | 0.10 | 0.80      |
| EXP-02      | Logistic Regression | 50   | 50   | 0.20 | 0.80      |
| EXP-03      | Random Forest       | 30   | 40   | 0.15 | 0.70      |

**Melhores hiperparâmetros encontrados:**

| Experimento | Hiperparâmetros otimizados                                                                     | Fitness (CV) |
|-------------|------------------------------------------------------------------------------------------------|--------------|
| EXP-01      | C=67.6702720428037, solver=lbfgs, max_iter=704, class_weight=balanced                          | 0.8786       |
| EXP-02      | C=2.104404966445881, solver=liblinear, max_iter=702, class_weight=None                         | 0.8796       |
| EXP-03      | n_estimators=352, max_depth=15, min_samples_split=3, min_samples_leaf=1, class_weight=balanced | 0.9579       |

### 3. Comparativo de Performance

Após otimização, cada conjunto de hiperparâmetros foi usado para treinar o modelo final e avaliado no conjunto de teste (holdout 20%, nunca visto durante a otimização).

| Modelo      | Experimento | Accuracy | Precision | Recall | F1     | ΔRecall |
|-------------|-------------|----------|-----------|--------|--------|---------|
| LR Baseline | —           | 83.37%   | 13.41%    | 44.00% | 0.2056 | —       |
| RF Baseline | —           | 92.86%   | 25.53%    | 24.00% | 0.2474 | —       |
| LR          | EXP-01      | 83.27%   | 12.88%    | 42.00% | 0.1972 | -2.00pp |
| LR          | EXP-02      | 83.17%   | 13.25%    | 44.00% | 0.2037 | +0.00pp |
| RF          | EXP-03      | 91.88%   | 18.87%    | 20.00% | 0.1942 | -4.00pp |

#### 3.1 Análise Crítica

**EXP-01** (LR, pop=20): convergiu em 4 gerações, encontrando  $C=67.67$  com `class_weight='balanced'`. Recall ficou em 42% — ligeiramente abaixo do baseline.

**EXP-02** (LR, pop=50): manteve recall de 44% (equivalente ao baseline). O AG confirmou  $C\approx 2.1$  com liblinear como região competitiva, diferente do EXP-01 ( $C$  alto com lbfgs), evidenciando múltiplos ótimos locais.

**EXP-03** (RF, pop=30): encontrou 352 árvores e `max_depth=15`, porém o recall do RF otimizado (20%) foi inferior ao baseline (24%), pois sem `class_weight='balanced'` o RF prioriza accuracy no dataset desbalanceado.

#### 3.2 Impacto Clínico

No conjunto de teste com 1.022 pacientes (50 casos positivos), recall de 44% detecta ~22 casos. Cada 10pp de melhora = 5 casos adicionais por lote de 1.022 pacientes — impacto significativo em escala hospitalar.

## 4. Integração com LLM para Interpretação de Resultados

### 4.1 Escolha do Modelo e Justificativa

- Gratuito para o volume do projeto (Free Tier da Google AI Studio)
- SDK oficial Python (google-generativeai) simples e bem documentado
- Sem necessidade de infraestrutura local (diferente de Llama/Ollama)
- Respostas de alta qualidade em português (PT-BR)

### 4.2 Estratégia de Prompt Engineering

**Prompt 1 — Diagnóstico Individual (build\_prediction\_prompt):** injeta features do paciente como bullets, instrui o modelo a usar apenas dados fornecidos, pede linguagem probabilística (não diagnóstico categórico) e resposta em PT-BR.

**Prompt 2 — Resumo Agregado (build\_experiment\_prompt):** injeta métricas baseline e otimizado, hiperparâmetros e deltas. Contextualiza recall como métrica principal e pede resumo executivo para gestores de saúde.

### 4.3 Avaliação da Qualidade das Interpretações

Checklist determinístico em `src/llm/evaluation.py` — roda em CI sem custo de API:

| Regra              | Verificação                                                        |
|--------------------|--------------------------------------------------------------------|
| mentions_recall    | Se <code>source_data</code> tem 'recall', o texto deve mencioná-lo |
| grounded_in_data   | Texto cita ao menos 1 valor numérico dos dados de entrada          |
| uses_risk_language | Usa palavras probabilísticas (não diagnóstico categórico)          |
| reasonable_length  | $40 \leq \text{len}(\text{texto}) \leq 4.000$ caracteres           |

Score final = média das 4 regras → float em [0, 1]. Exemplos de saída completos estão no notebook `notebooks/03_llm_integration.ipynb`.

## 5. Desafios Enfrentados e Soluções

---

**Custo computacional do AG:** Com CV de 5 folds e pop=50, cada experimento demorava >15 minutos. Solução: CV=2 folds + early stopping agressivo (patience=4), reduzindo para ~2 minutos.

**Dataset fortemente desbalanceado (4.9% positivos):** O AG tendia a otimizar accuracy ignorando recall. Solução: penalização explícita na fitness para recall < 0.30.

**Convergência rápida do AG para LR:** O espaço de busca da LR é relativamente plano — convergência em 4 gerações. Para exploração mais ampla, populações maiores ou mutação mais agressiva seriam necessárias.

**Qualidade das explicações do LLM:** LLM-as-judge introduziria viés e custo. Optamos por checklist determinístico de 4 regras, reproduzível em CI, com a limitação de não capturar nuances semânticas finas.

**Encoding de notebooks gerados programaticamente:** Notebooks gerados por script Python tinham encoding incorreto. Solução: reescrever células markdown com `json.dump()` explícito em UTF-8.

## 6. Conclusão

---

Este projeto demonstrou a aplicação prática de Algoritmos Genéticos para otimização de hiperparâmetros em diagnóstico médico, complementada por LLM para tornar os resultados acessíveis a profissionais de saúde.

- EXP-02 manteve recall de 44% (equivalente ao baseline), validando a abordagem
- LR é eficientemente explorada com populações de 20–50 indivíduos
- RF é mais sensível à configuração de `class_weight` — trabalhos futuros devem incluí-lo
- Integração com Gemini produziu explicações coerentes e grounded nos dados
- Suíte de 54 testes automatizados garante robustez do pipeline completo

### 6.1 Próximos Passos (Fase 3)

- Explorar espaços de busca maiores (XGBoost, redes neurais) com AG multiobjetivo
- Integrar dados textuais via embeddings para enriquecer as features
- Monitoramento de drift para detectar degradação do modelo em produção
- Avaliação clínica especializada das explicações do LLM

Repositório: <https://github.com/brunosouza2/stroke-prediction>